

Project 3: Campus Compass: Documentation

What is the time complexity of each method (corresponding to a command) in your implementation? Reflect on the worst-case time complexity represented in Big O notation. [10 points]

The following variables are used in discussing the time complexity of the methods of the implementation.

Variable	Definition
a	Length of the longest students name.
b	length of the longest residence ID.
c	number of unique residence IDs in the validResidences set.
d	number of students in the students map.
e	number of courses in the catalog map.
f	number of students in a given course's roster.
g	maximum edges in a node in the adjacency list.
h	number of nodes in the adjacency list.
m	total number of edges in the edgeList.
p	number of location-ID pairs supplied as N.

insert

This method has a time complexity of $O(a + b + \log(c) + \log(d) + \log(e) + f)$ – In this method we call the helper to validate the name, UFID, and Residence ID input the worst case for these is the longest name (a), UFID (8), and longest residence ID (b). We then have to perform a lookup in both the students map and the valid residences set both of which are ordered and thus operations are determined by the log of their size ($\log(d) + \log(c)$). We then perform a lookup in the catalog map for each course supplied to confirm it exists this map is ordered ($\log(e)$). Lastly, for each course added a linear search of the courses' roster is performed to ensure the student isn't already enrolled (f).

remove

This method has a time complexity of $O(\log(d) + \log(e) + f)$ - In this method we validate the UFID input, then call the removeStudent helper. This performs a lookup in the students map ($\log(d)$). We then iterate over the student's course schedule and for each course perform a lookup in the catalog map ($\log(e)$), followed by a linear search of that course's roster to remove the student (f).

dropClass

This method has a time complexity of $O(\log(d) + \log(e) + f)$ - In this method we validate the UFID input, then call the dropStudent helper. This performs lookups in both the students map and catalog map ($\log(d) + \log(e)$), followed by a linear search of the specified course's roster to remove the student (f).

replaceClass

This method has a time complexity of $O(\log(d) + \log(e) + f)$ - In this method we validate the UFID input, then call the swapStudent helper. This performs lookups in both the students map and catalog map ($\log(d) + \log(e)$), then performs two linear searches of course rosters — one to remove the student from the old course and one to add them to the new course — both bounded by the size of the roster (f).

removeClass

This method has a time complexity of $O(\log(e) + f * \log(d))$ - In this method we perform a lookup in the catalog map to confirm the course exists ($\log(e)$). We then iterate over every student in that course's roster (f), and for each one perform a lookup in the students map to remove the course from their schedule, another ordered structure ($\log(d)$) — since this lookup happens once per student in the roster rather than once overall, this term is a product rather than a sum ($f * \log(d)$).

toggleEdgesClosure

This method has a time complexity of $O(p * (b + \log(c) + \log(h) + g^2))$ - In this method we validate each of the p location-ID pairs supplied, the worst case for the ID length being the longest residence ID (b), and each validation call also performs a lookup in the valid residences set ($\log(c)$). For each pair we then perform a lookup in the adjacency list ($\log(h)$), and perform a nested linear search across both locations' edge lists to find the shared edge, each of which can hold up to g edges (g^2). Since this work repeats once per pair, the entire expression is multiplied by p .

checkEdgeStatus

This method has a time complexity of $O(\log(h) + g^2)$ - In this method we perform a lookup for each location in the adjacency list ($\log(h)$), and then perform a nested linear search across both locations' edge lists to find whether a shared edge exists between them, each of which can hold up to g edges (g^2).

isConnected

This method has a time complexity of $O((h+m) * \log(h))$ - In this method we perform a breadth-first search across the graph. In the worst case, every node is visited (h) and every edge is examined (m), and because both the adjacency list and the visited set are ordered structures, each of these operations is determined by the log of the number of nodes ($\log(h)$).

printShortestEdges

This method has a time complexity of $O(\log(d) + \log(e) + m \cdot \log(h) + m \cdot \log(m))$ - In this method we perform a lookup in the students map to confirm the student exists ($\log(d)$). We then call the Dijkstra helper to compute shortest distances from the student's residence across the entire graph, in the worst case this examines every edge (m), with each relaxation performing a lookup in the ordered distances map ($\log(h)$) and a push/pop on the priority queue ($\log(m)$). Finally, for each course in the student's schedule we perform a lookup in the catalog map ($\log(e)$).

printStudentZone

In this method we perform a lookup in the students map, an ordered structure ($\log(d)$), and then call the Dijkstra helper, which in the worst case examines every edge in the graph (m), with each relaxation performing lookups determined the number of nodes and edges ($\log(h) + \log(m)$). We then reconstruct the shortest path to each course location, which in the worst-case walks through every node in the graph (h), with each step performing a lookup on an ordered structure ($\log(h)$). Finally, we collect candidate edges and sort them, then build the minimum spanning tree by processing each edge in order of weight, for each edge, we use a disjoint set to look up which group each of its two locations belongs to, and only keep the edge if the two locations are not already in the same group, otherwise adding it would create a cycle, so it's skipped. Each of these group lookups is based on the number of nodes ($\log(h)$), and this is repeated across up to m candidate edges.

verifySchedule

This method has a time complexity of $O(\log(d) + \log(e) + m \cdot \log(h) + m \cdot \log(m))$ - In this method we perform a lookup in the students map to confirm the student exists ($\log(d)$). We then sort the students' schedule by start time, with each comparison performing a lookup in the catalog map ($\log(e)$). For each adjacent pair of courses in the schedule, we call the Dijkstra helper to check reachability within the available time buffer, in the worst case this examines every edge in the graph (m), with each relaxation performing lookups on ordered structures determined by the number of nodes and edges ($\log(h) + \log(m)$). Since the number of courses is at most six, we can ignore the number of comparisons made.

What did you learn from this assignment, and what would you do differently if you had to start over? [1 point]

I originally brainstormed and took the ideation phase from bottom up beginning with the simplest pieces and working my way up to the finalized graph structure. What this ended up with was the four distinct objects that made up my final structure. The problem I found in doing this is that I too quickly transitioned into real code prior to full ideation on the methods and implementation, this led to multiple restructuring phases of both the original underlying objects and the graph implementation entirely. Originally had the graph structure as strictly run through a 2-d vector adjacency list these changes twice until I ended up as my current state and both changes led to rewrites of all underlying object attributes and methods. Development would have been drastically quicker if I did not have to make such changes and it would have made for simpler and much more readable functionality as in doing these restructures non-ideal implementations were made to reduce the friction of having to restructure the entire set of objects. Additionally, I did not gradually develop testing for my structure and therefore after the majority of the project had been developed I had to break it apart to test it and debug, in future projects I would like to slow down with implementation and take time between each implementation phase to test and debug the implementation rather than testing it only near the end of implementation leading to spending the majority of the time testing slowly crawling through methods to debug small issues.

Write 2-4 sentences on the project difficulty, intuition, and your overall thoughts on the project. [1 point]

I felt confident in choosing the appropriate underlying graph structure for this project, seeing that it was a sparse graph an adjacency list was intuitive. However, it was difficult to ideate on parsing this structure efficiently especially when edges needed to be linked to ensure proper closure on an undirected graph system. The most challenging portion of this project was handling the expected input and passing it back and forth through the objects. I still believe that my method of logging roster on both course and student objects is intuitive and easy to follow but not optimal, as well as dual handling of edges in my partner edge and adjacency lists both felt like they could be improved further.